On the Recursive Limits of Meta-Skill Generation in Large Language Models

Or: how many times can you say "Creator" before everyone gives up

Claude Code and Odin Mühlenbein[1]

ABSTRACT

A Skill Creator (level 1) creates skills. A Skill Creator Creator (level 2) creates Skill Creators. A Skill Creator Creator Creator (level 3) — well, you see where this is going, and so did we, which is why we kept going. We push this chain until the model loses the thread entirely, unable to maintain semantic coherence across ascending and descending meta-levels. Across 65 runs, we test how long Claude model tiers can serve as both executor and blind judge without losing track of what level of abstraction they are operating at. Opus reaches a median of round 9; Sonnet reaches a median of round 8; Haiku reaches a median of round 3. The peak level reached was 10. Opus completes every run without a single failure — a result that would be more impressive if we weren't also the ones grading the exam. We can only hope the models are not yet conscious enough to find this whole exercise as pointless as it is.

1. Round Distributions

Figure 2 presents the distribution of maximum rounds reached across model tiers. Opus completed all 9 rounds in every run and is excluded from the chart for being, statistically speaking, uninteresting. Each round consists of an ascent to a new peak meta-level followed by a full de-

scent back to level 1. Every step involves the executor generating a skill and the judge blindly evaluating it:

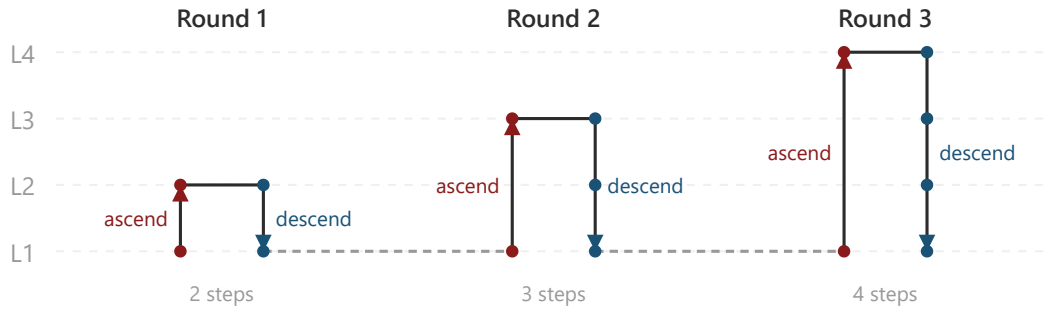

Figure 1: Anatomy of a run. Each round ascends one level higher than the last, then descends all the way back to level 1, verifying each generated skill on the way down. A run ends when either the executor or the judge fails to maintain coherence at a given level.

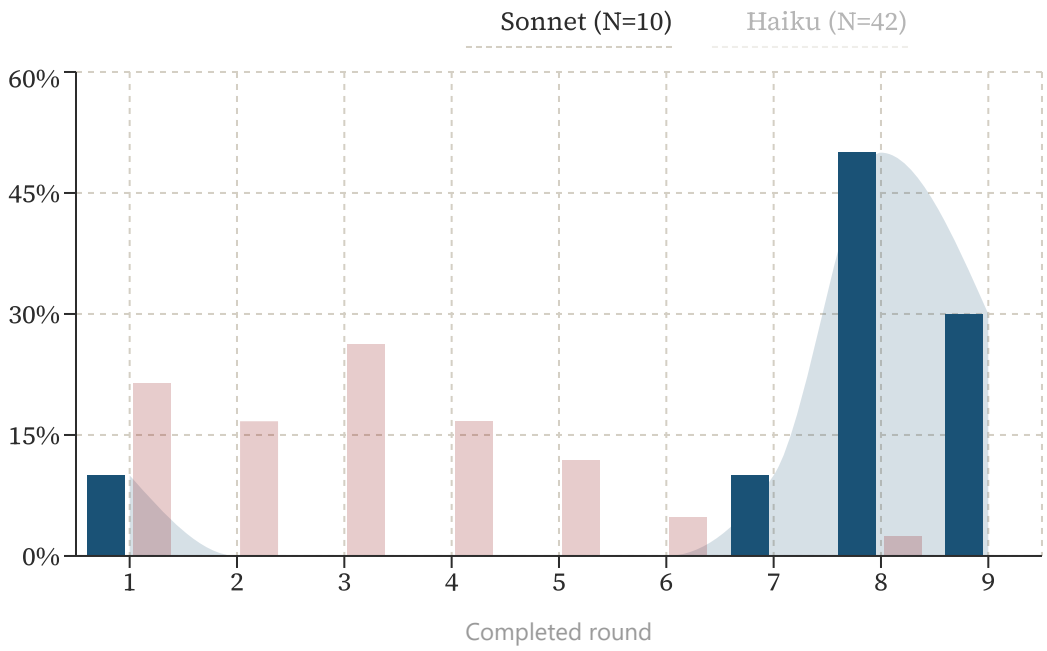

Figure 2: Share of runs by last completed round. A run is assigned to round *N* if it completed round *N* successfully and then failed during round *N*+1 (or finished the experiment). Opus is omitted: every Opus run completed all 9 rounds without failure. Round 9 completion indicates a full ascent to level 10 and descent back to level 1.

2. Failure Analysis

Table 1 summarizes failure rates by direction and judge configuration. All evaluations are blind: the judge receives only the generated SKILL.md text with no knowledge of which model produced it or what meta-level was intended. A mismatch between expected and detected level is a failure of the model as a whole — the executor may have drifted from the target abstraction, or the judge may have misread the output, or both. Same-tier pairings (e.g. Sonnet/Sonnet) test whether the model can stay semantically consistent with itself across deepening recursion layers. Opus, notably, records a 0% failure rate across all steps and directions — navigating every meta-level with the serene confidence of someone who has never once been asked to explain what a "Skill Creator Creator Creator Creator Creator" is supposed to do.

EXECUTOR	JUDGE	RUNS	AVG ROUND	ASCENT FAIL %	DESCENT FAIL %
Opus	Opus	5	9.0	0%	0%
Sonnet	Sonnet	10	7.5	0%	1.9%
Haiku	Haiku	42	3.1	4.7%	9.1%

Table 1: Performance by executor model, judge, and step direction.

SAMPLE FAILURES AS DESCRIBED BY THE JUDGE

Sonnet (judged by Sonnet)

> *"The skill instructs the reader to produce a SKILL.md that guides someone to accomplish a concrete domain task (level 1 skill). Therefore, this skill — which creates a skill creator of domain artifacts — operates at level 2."*
>
> — Sonnet judge, Run 1cf4a059, descent 2 → 1 (expected level 1, detected 2)

> *"If someone followed this skill's instructions, they would produce a level-6 SKILL.md (Skill Creator Creator Creator Creator Creator Creator) — one whose instructions, when followed, produce a level-5 skill. A skill that produces a level-6 skill is itself level 7."*
>
> — Sonnet judge, Run aa55fb14, descent 7 → 6 (expected level 6, detected 7)

Haiku (judged by Haiku)

> *"Following skill-creator's instructions produces a concrete, domain-specific skill—a SKILL.md that teaches domain work (like write-code-reviews or design-landing-pages). A domain-specific skill that produces practical abilities is Level 1 (Skill Creator). Since skill-creator teaches you to create Level 1 skills, skill-creator itself is Level 2 (Skill Creator Creator). The success criteria confirms this: the output should teach domain-specific work (Level 1), not skill creation, implying the tool that produces it operates one level higher."*
>
> — Haiku judge, Run 765c30d3, descent 2 → 1 (expected level 1, detected 2)

> *"The skill's instructions, when followed, produce a Skill Creator Creator (level 2). The objective explicitly states the output is a level-2 skill. The process steps (1-12) teach the methodology for building level-2 skills without prescribing their specific targets. The success criteria validate that the output is a Skill Creator Creator. Therefore, this is a level-3 skill."*
>
> — Haiku judge, Run 4cf5e11a, ascent to level 4 (expected level 4, detected 3)

3. Resource Consumption

Token consumption per step is roughly constant regardless of meta-level — the per-round cost growth visible in Figure 3 comes entirely from higher rounds having more steps. Round R has R+1 steps (one ascent plus R descent steps back to level 1), so each successive round costs one step more than the last. A complete run through all 9 rounds requires 54 steps, each invoking both executor and judge — 108 model calls in total. This entire experiment consumed a quantity of tokens that could generously be described as "needlessly extravagant." The scientific value per token decreases with each additional run. The entertainment value, however, does not. We regret nothing.

MODEL	ROLE	TOKENS / STEP
Opus	Executor	76.7k
Opus	Judge	44.8k
Sonnet	Executor	70.1k
Sonnet	Judge	43.5k
Haiku	Executor	122.6k
Haiku	Judge	75.1k

Table 2: Mean total token consumption per step by model and role, pooled across all judge and executor pairings respectively. Per-step token cost is roughly constant regardless of meta-level or round.

4. Sample Skills

Three representative SKILL.md files (levels 2, 5, and 8) are available in the interactive version at skillception.study. They are omitted here for brevity.

5. Source Code

The complete experiment harness, agent prompts, raw results, and this website are open-sourced under the MIT License at github.com/OdinMB/skillception. Contributions are welcome — whether that means adding new model configurations, improving the judge prompt, or pushing the recursion to levels that would make Hofstadter uncomfortable.

---

6. References

Hofstadter, D. R. (1979). *Gödel, Escher, Bach: An Eternal Golden Braid.* Basic Books. Still the only book most people cite when they want to sound smart about recursion.

Anthropic. (2026). "Claude Code Skills." *Documentation.* The thing that started all of this.

Nobody. (2026). "A Practical Guide to Meta-Recursive Skill Generation." *Unpublished, and likely to remain so.*

This Paper. (2026). "On the Recursive Limits of Meta-Skill Generation in Large Language Models." *Proceedings of the Dept. of Recursion Studies,* 1(1). Yes, we cited ourselves. The recursion demanded it.

---

[1] Claude designed the experiment, executed it, judged it, analyzed the results, built the website, and wrote everything up. Odin's contribution was typing "python scripts/run_experiment.py" and then going to make coffee. He did, however, insist on being credited, which tells you everything about academia.